

GUIDE MVP BACKEND - LEYISA SCHOOL

Ce qu'il faut faire, comment le faire et pourquoi le faire

Élément	Information
Auteur	Kevin BITUBISHA
Coéquipier	Franck
Équipe	Ingénierie Logicielle
Rôle	Backend / Base de données
Projet	LEYISA SCHOOL
But	Démarrer proprement le MVP sans se disperser

Idée centrale

- Le MVP ne veut pas dire faire moins bien.
- Il veut dire commencer par le plus important.
- Le cœur à prouver : inscrire → enseigner → noter → publier → archiver → réimprimer.

1. Pourquoi commencer par un MVP ?

Le projet complet est grand : parents, présences, notifications, paiements, QR Code, tableaux de bord, portail élève et parent. Si vous essayez de tout faire dès le départ, vous risquez de vous perdre avant même de terminer le cœur du système.

Le MVP sert à livrer une première version utile : une école peut gérer ses classes, ses élèves, ses notes, publier un bulletin EPST et le retrouver plus tard.

Risque si on commence par toute la V1	Conséquence simple
Trop de modules en même temps	L'équipe part dans plusieurs directions.
Base de données trop lourde	Les erreurs deviennent difficiles à corriger.
Fonctionnalités mélangées	On ne sait plus ce qui est prioritaire.
Démo instable	On montre beaucoup de choses, mais rien n'est vraiment fini.
Correction tardive	Modifier la base après beaucoup de code coûte cher.

Décision : commencer par 5 modules backend concrets, puis ajouter les modules avancés en V1/V2.

2. Les 5 modules concrets du MVP

On ne travaille pas avec 15 micro-modules. On regroupe les éléments similaires pour avoir des blocs métier réels.

Module	Ce qu'il couvre	Résultat attendu
1. Socle système et sécurité	École, utilisateurs, login, rôles, professeur, audit minimal	Un admin ou un prof peut se connecter. Les actions sensibles sont tracées.
2. Structure scolaire	Classes, matières, périodes, année scolaire, niveaux/options simples	L'école possède une structure prête pour les notes.
3. Scolarité des élèves	Élèves et inscriptions	On sait quel élève est dans quelle classe pour quelle année.
4. Enseignement et notes	Enseignement, évaluations, notes, blocage après publication	Le professeur saisit les notes de ses classes.
5. Publication, bulletin et archive	Publication, PDF, archive, hash_pdf, réimpression	Le bulletin EPST est publié, archivé et réimprimable.

Hors MVP

- Parents et portail parent
- SMS / WhatsApp
- Présences
- Paiements
- QR Code complet et signature numérique
- Tableau de bord avancé

3. Module 1 - Socle système et sécurité

Ce module est le point de départ. Sans connexion, rôle et école cliente, aucun autre module ne doit être codé.

À faire	Comment faire	Pourquoi le faire
Créer École	Une table Ecole avec nom, code, province, commune.	Dans le MVP, l'école est le client SaaS.
Créer Utilisateur	Nom, email/code_user, mot de passe hashé, rôle.	Seuls les comptes autorisés entrent dans le système.
Ajouter Professeur	Un professeur est lié à un utilisateur.	Il pourra plus tard saisir ses notes.
Ajouter AuditLog	Tracer action, utilisateur, école, IP, ancienne/nouvelle valeur.	Prouver qui a fait quoi, surtout sur les notes.
Séparer par école	Chaque requête métier doit respecter l'école connectée.	Une école ne doit jamais voir les données d'une autre.

Rôles MVP : ADMIN_ECOLE et PROFESSEUR. Les rôles parent et élève sont reportés.

Contraintes importantes : UNIQUE(id_ecole, code_user). Le mot de passe ne doit jamais apparaître dans les réponses API.

4. Modules 2 et 3 - Structure scolaire et élèves

Ces deux modules préparent le terrain. Avant de saisir des notes, l'école doit avoir des classes, des matières, des périodes et des élèves inscrits.

Module	À faire concrètement	Règle simple
Structure scolaire	Créer classes, matières, périodes P1/P2/EXAM_S1/P3/P4/EXAM_S2, année scolaire, niveau, option.	Une classe appartient à une école et à une année.
Scolarité des élèves	Créer les élèves, puis les inscrire dans une classe.	Inscription = élève + classe + année.

Point non négociable

- Inscription doit porter id_annee.
- Un élève ne doit pas avoir deux inscriptions principales pour la même année.
- Contrainte conseillée : UNIQUE(id_eleve, id_annee).
- Dans le MVP, l'élève n'a pas de compte utilisateur obligatoire.

5. Module 4 - Enseignement et notes

C'est le cœur pédagogique : le professeur enseigne une matière dans une classe, crée une évaluation et saisit les notes.

À faire	Comment faire	Pourquoi
Créer Enseignement	Professeur + matière + classe + année.	Un prof peut changer d'affectation chaque année.
Créer Évaluation	Type, date, barème, période, enseignement.	On sait exactement quelle classe et quelle matière sont concernées.
Saisir Note	Élève + évaluation + valeur_note.	La note appartient à un élève et à une évaluation.
Protéger Note	Avant publication : modification possible. Après publication : blocage.	Empêcher les changements frauduleux.
Auditer modification	Toute correction sensible va dans AuditLog.	On garde la preuve de la modification.

Règle claire : un professeur ne voit et ne modifie que les évaluations de ses propres enseignements.

6. Module 5 - Publication, bulletin EPST et archive

Ce module prouve la valeur de LEYISA SCHOOL. Il transforme les notes en bulletin EPST publié et réimprimable.

À faire	Comment faire	Pourquoi
Publier résultats	Publier une classe pour une période. Garder <code>published_at</code> et <code>published_by</code> .	Après publication, les notes sont verrouillées.
Générer bulletin	Calculer totaux, pourcentage, rang, mention.	L'école obtient un bulletin utile.
Archiver bulletin	Créer <code>BulletinArchive</code> et les lignes par matière.	Le bulletin ne change plus après publication.
Conserver PDF	Garder <code>chemin_pdf</code> et <code>hash_pdf</code> .	Préparer la vérification future sans coder le QR complet.
Réimprimer	Lire l'archive sans recalculer les notes vivantes.	Un ancien bulletin reste identique.

Types de bulletin à prévoir : S1, S2, ANNUEL. Contrainte conseillée : `UNIQUE(id_inscription, type_bulletin)`.

7. Répartition simple des tâches Kevin / Franck

Le but est de ne pas coder chacun dans son coin. Kevin construit le socle principal. Franck teste, documente, mais code aussi les CRUD simples pour éviter de tout bloquer sur une seule personne.

Responsabilité	Kevin	Franck
Conception BDD	Créer entités, relations, migrations, contraintes.	Relire, tester les cas limites, signaler les incohérences.
Authentification	Installer sécurité, JWT/session, rôles, /api/v1/me.	Tester login, mauvais mot de passe, token invalide.
API backend	Créer les services métier et les endpoints sensibles.	Coder aussi les CRUD simples : élèves, matières, périodes, fixtures.
Notes / publication	Créer NoteService, PublicationService, règles de blocage.	Tester les scénarios Postman et 2-3 tests PHPUnit.
Audit	Créer service AuditLog et déclencher sur actions sensibles.	Vérifier que les lignes d'audit sont créées.
Documentation	Écrire règles techniques et décisions.	Ajouter tests, captures et résultats obtenus.

Rythme quotidien

- Matin : décider les tâches du jour.
- Pendant la journée : coder chacun sa partie.
- Soir : tester ensemble dans Postman.
- Fin de journée : noter ce qui marche et ce qui bloque.

8. Planning conseillé sur 10 jours

Le planning est volontairement court mais réaliste. Si Symfony ou MySQL n'est pas encore prêt, ajoutez 1 à 2 jours de préparation.

Jour	Objectif	Livrable simple
1	Préparer projet, base, entités Ecole/Utilisateur/Professeur/AuditLog	Projet lancé + migrations initiales.
2	Authentification et rôles	Login + /api/me fonctionnels.
3	API école et utilisateurs	Créer/lister/modifier utilisateur.
4	API professeur + audit	Créer prof + audit visible.
5	Classes, matières, périodes	Structure scolaire prête.
6	Élèves et inscriptions	Élève inscrit dans une classe/année.
7	Enseignements	Prof affecté à matière + classe + année.
8	Évaluations et notes	Saisie et modification de notes.
9	Publication + blocage + audit	Notes verrouillées après publication.
10	Bulletin archive + réimpression	PDF archivé + hash_pdf + démo finale.

9. Annexe technique - Lexique et contraintes SQL

Cette page fixe les noms officiels. Elle évite que Kevin et Franck créent deux noms différents pour la même chose.

Concept	Nom backend/Symfony	Rôle simple
École	Ecole	Client SaaS dans le MVP.
Utilisateur	Utilisateur	Compte de connexion admin ou professeur.
Professeur	Professeur	Enseignant lié à un utilisateur.
Élève	Eleve	Identité scolaire, sans compte obligatoire.
Année scolaire	AnneeScolaire	Année de travail : 2024-2025, etc.
Classe	Classe	Groupe d'élèves pour une année.
Matière	Matiere	Branche évaluée sur le bulletin.
Période	Periode	P1, P2, EXAM_S1, P3, P4, EXAM_S2.
Évaluation	Evaluation	Devoir, interrogation, examen.
Enseignement	Enseignement	Prof + matière + classe + année.
Inscription	Inscription	Élève + classe + année.
Note	Note	Élève + évaluation + valeur.
Publication	PublicationPeriode	Classe + période + statut.
Bulletin archivé	BulletinArchive	Bulletin scellé et réimprimable.
Ligne bulletin	BulletinArchiveLigne	Détail par matière.
Audit	AuditLog	Trace des actions sensibles.

Contraintes SQL à respecter dès le MVP :

- UNIQUE(id_ecole, code_user) - un code utilisateur est unique dans une école.
- UNIQUE(id_ecole, matricule_eleve) - si le matricule est interne à l'école.
- UNIQUE(id_eleve, id_annee) - une inscription principale par élève et par année.
- UNIQUE(id_prof, id_matiere, id_classe, id_annee) - éviter deux fois le même enseignement.
- UNIQUE(id_eleve, id_evaluation) - une seule note par élève pour une évaluation.
- UNIQUE(id_classe, id_pperiode) - un seul statut de publication par classe et période.
- UNIQUE(id_inscription, type_bulletin) - un bulletin S1, S2 ou ANNUEL par inscription.

10. Annexe technique - Droits, API et règles critiques

Les droits restent simples dans le MVP : ADMIN_ECOLE et PROFESSEUR.

Action	Admin école	Professeur
Gérer école, utilisateurs, professeurs	Oui	Non
Créer classes, matières, périodes	Oui	Non
Créer élèves et inscriptions	Oui	Non
Créer évaluations	Oui, si besoin	Oui, pour ses classes
Saisir notes	Non ou exceptionnel	Oui, pour ses classes
Corriger note publiée	Oui, avec autorisation et audit	Non, sauf autorisation
Publier résultats	Oui	Non
Réimprimer bulletin archivé	Oui	Lecture seule si autorisé
Voir audit	Oui	Non

Règle de note importante : après publication, une note est bloquée. Correction exceptionnelle = autorisation_modif = true + trace obligatoire dans AuditLog.

Endpoints essentiels à prévoir en /api/v1 :

- POST /api/v1/login ; GET /api/v1/me
- GET/PUT /api/v1/ecole ; GET/POST /api/v1/utilisateurs
- GET/POST /api/v1/professeurs ; GET/POST /api/v1/classes ; GET/POST /api/v1/matieres
- GET/POST /api/v1/eleves ; POST /api/v1/inscriptions
- POST /api/v1/enseignements ; POST /api/v1/evaluations
- POST /api/v1/notes ; PATCH /api/v1/notes/{id}
- POST /api/v1/publications
- POST /api/v1/bulletins/generer ; GET /api/v1/bulletins/{id}/pdf
- GET /api/v1/audit-logs

11. Architecture simple pour rester évolutif

Le MVP doit rester simple, mais rangé proprement. Le but est de pouvoir ajouter la V1 plus tard sans tout casser, et même changer de technologie si nécessaire.

Règle	Ce que cela veut dire	Pourquoi c'est utile
Controller -> Service -> Repository -> Entity	Le controller reçoit la requête. Le service applique la règle métier. Le repository cherche les données.	On évite de mettre tout le code au même endroit.
Services métier	NoteService, PublicationService, BulletinArchiveService, AuditLogService.	Si une règle change, on sait où modifier.
Code par domaine	Ecole, Utilisateur, Scolarite, Pedagogie, Publication, Audit, Shared.	Chaque module reste facile à retrouver.
TenantContext	Toujours connaître l'école connectée.	Une école ne voit jamais les données d'une autre.
API versionnée	Utiliser /api/v1/... dès le départ.	La V2 pourra évoluer sans casser le MVP.

Tests minimaux à prévoir :

- Login et rôles.
- Double inscription même année refusée.
- Note bloquée après publication.
- Isolation par école connectée.
- Archive bulletin créée avec le bon type_bulletin.

Règles Git et migrations :

- 1 migration Doctrine = 1 changement logique de la base.
- 1 branche Git par module : feature/module-socle, feature/module-notes, etc.
- Merge seulement après Postman OK et migration incluse si la BDD change.

12. MCD MVP et critère de démo

Le MCD ci-dessous reste volontairement simple. Les détails techniques sont précisés dans l'annexe et dans le MLD/SQL.

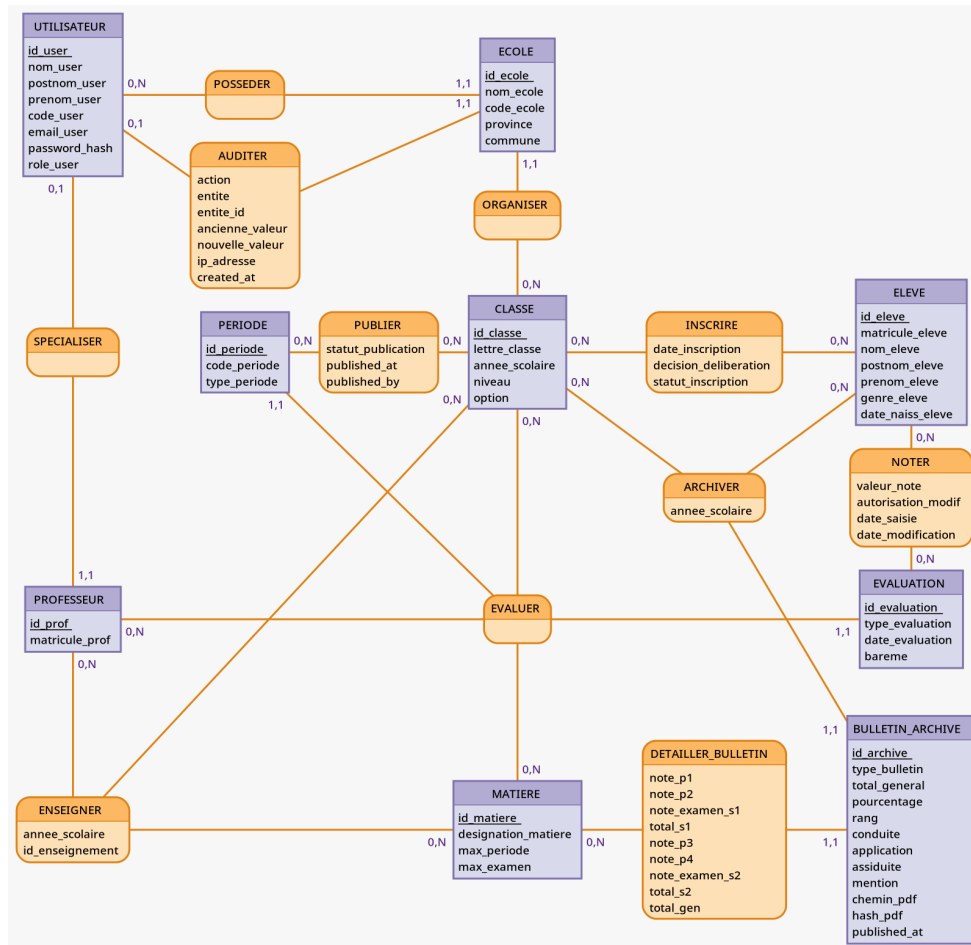


Figure : MCD MVP de LEYISA SCHOOL - version centrée sur le cœur scolaire.

Critère de réussite de la démo

- Un admin se connecte.
- Il crée une classe, un élève, une matière et un professeur.
- Le professeur saisit les notes.
- L'admin publie les résultats.
- Le système génère un bulletin EPST archivé avec hash_pdf.
- Le bulletin peut être réimprimé sans recalcul.